第18章：我们学到了什么

五个架构赌注

Claude Code 不是唯一的代理系统。它不是第一个。但它做了五个架构赌注，使其与代理框架领域区分开来，经过近两千个文件和十七章之后，这些赌注值得审视。

赌注 1：生成器循环优于回调

大多数代理框架给你一个管道：定义工具，注册处理器，让框架编排。开发者编写回调。框架决定何时调用它们。

Claude Code 做相反的事情。`query()` 函数是一个异步生成器——开发者拥有循环。模型流式传输响应，生成器产生工具调用，调用者执行它们，追加结果，生成器循环。有一个函数，一个数据流，一个每个交互都经过的地方。生成器返回类型的 10 个终止状态和 7 个继续状态编码了每个可能的结果。循环就是系统。

赌注是单个生成器函数，即使它增长到 1,700 行，也会比分布式回调图更易理解。研究源代码后，赌注得到了回报。当你想知道会话为什么结束时，你看一个函数。当你想添加新的终止状态时，你向一个可辨识联合添加一个变体。类型系统强制执行穷尽处理。回调架构会将此逻辑分散在几十个文件中，回调之间的交互将是隐式的而不是在控制流中可见的。

赌注 2：基于文件的内存优于数据库

第11章详细阐述了这一点，但架构意义超越内存。使用纯 Markdown 文件而不是 SQLite、向量数据库或云服务的决定是透明性胜过能力的赌注。数据库将支持更丰富的查询、更快的查找和事务保证。文件不提供这些。文件提供的是信任。

在 vim 中打开 `~/.claude/projects/myapp/memory/MEMORY.md` 并准确看到代理关于他们记住什么的用户，与必须问代理"你记得什么？"并希望答案完整的用户，与系统有着根本不同的关系。基于文件的设计使代理的知识状态外部可观察，而不仅仅是自我报告的。这比查询性能更重要。LLM 驱动的回忆系统用检索智能补偿存储简单性——一个 Sonnet 副查询从清单中选择五个相关记忆比嵌入相似性更精确，且需要零基础设施。

赌注 3：自描述工具优于中央编排器

代理框架通常提供工具注册表：你在中央配置中描述你的工具，框架将它们呈现给模型。Claude Code 的工具自我描述。每个 `Tool` 对象携带自己的名称、描述、输入模式、提示贡献、并发安全标志和执行逻辑。工具系统的工作不是向模型描述工具——而是让工具自我描述。

这个赌注在可扩展性上得到回报。MCP 工具（第15章）通过实现相同接口成为一等公民。来自 MCP 服务器的工具和内置工具对模型来说无法区分。系统不需要单独的"MCP 工具适配器"层——包装产生标准 `Tool` 对象，从该点起，现有工具管道处理它：权限检查、并发执行、结果预算、钩子拦截。

赌注 4：Fork 代理用于缓存共享

第9章涵盖了 fork 机制：一个以父代完整对话开始其上下文窗口的子代理，共享父代的提示缓存。这不是一个便利优化——而是缓存共享模型值得 fork 生命周期管理复杂性的架构赌注。

替代方案——用对话摘要生成一个全新的代理——更简单但昂贵。每个新代理都支付从头处理其上下文的全部成本。fork 的代理免费获得父代的缓存前缀（输入令牌 90% 折扣），使其对小任务生成代理经济：记忆提取、代码审查、验证传递。后台记忆提取代理（第11章）在每个查询循环轮次后运行，其成本是微不足道的，正是因为它共享父代的缓存。没有基于 fork 的缓存共享，该代理将成本高得令人望而却步。

赌注 5：钩子优于插件

大多数可扩展性系统使用插件——注册能力并在宿主进程内运行的代码。Claude Code 使用钩子——在生命周期点运行的外部进程，通过退出码和 stdin/stdout 上的 JSON 通信。

赌注是进程隔离值得开销。插件可能使宿主崩溃。钩子只崩溃自己的进程。插件可能将内存泄漏到宿主的堆中。钩子的内存随其进程死亡。插件需要必须版本化和维护的 API 表面。钩子需要 stdin、stdout 和退出码——自 1971 年以来一直稳定的协议。

开销是真实的：每个钩子调用生成一个进程花费毫秒，而进程内回调不会。内部回调的 -70% 快速路径（第12章）表明系统知道这种成本很重要。但对于外部钩子——用户脚本、团队 linter、企业策略服务器——隔离保证使系统更安全地扩展。企业可以部署基于钩子的策略强制执行，而不用担心格式错误的钩子脚本会崩溃其开发者的会话。

什么可以迁移，什么不可以

不是 Claude Code 中的每个模式都能泛化。有些是规模、资源或特定约束的后果，其他代理构建者可能不共享。

可迁移到任何代理的模式

生成器循环模式。 任何需要流式响应、处理工具调用和管理多个终止状态的代理都受益于使循环显式而不是将其隐藏在回调后面。可辨识联合返回类型——编码循环停止的确切原因——是一种消除整个"代理为什么停止？"调试会话类别的模式。

基于文件的内存与 LLM 回忆。 具体实现细节是 Claude Code 的，但原则——简单存储结合智能检索——适用于任何需要跨会话持久化知识的代理。四类型分类法（用户、反馈、项目、参考）和可推导性测试（"这能从当前项目状态重新推导吗？"）是可重用的设计启发式。

远程执行的不对称读/写通道。 当读取是高频率流而写入是低频率 RPC 时，分离它们是正确的，无论具体的传输协议如何。

搜索的位图预过滤器。 任何搜索大型文件索引的代理都受益于作为预过滤器的 26 位字母位图。每条目 4 字节，每个候选者一次整数比较——成本收益比显著。

提示缓存稳定性作为架构关注点。 如果你的代理使用带提示缓存的 API，将提示构建为稳定内容在前、易变内容在后不是优化——而是决定你成本结构的架构决策。

Claude Code 规模特定的模式

Fork 的终端渲染器。 Claude Code fork 了 Ink 并用打包的类型化数组、基于池的内部化和单元格级差异比较重新实现了渲染管道，因为它需要终端中的 60fps 流式传输。大多数代理渲染到网页界面或简单日志输出。只有当终端渲染是你的主要 UI 且你以高频率流式传输时，工程投资才有意义。

50 多个启动分析检查点。 当你有数十万用户且 0.5% 采样产生统计显著数据时才有意义。对于较小的代理，更简单的计时系统就足够了。

八种 MCP 传输类型。 Claude Code 支持 stdio、SSE、HTTP、WebSocket、SDK、两个 IDE 变体和 Claude.ai 代理，因为它必须与每种部署拓扑集成。大多数代理需要 stdio 和 HTTP。

钩子快照安全模型。 在启动时冻结钩子配置且不再隐式重新读取它是对特定威胁的防御：恶意仓库代码在用户接受信任对话框后修改钩子。当你的代理在具有不受信任 `.claude/` 配置的任意仓库中运行时，这很重要。只在受信任环境中运行的代理可以使用更简单的钩子管理。

复杂性的成本

近两千个文件。这买了什么，又花了什么？

文件计数作为复杂性指标是误导性的。其中大部分是测试基础设施、类型定义、配置模式和 fork 的 Ink 渲染器。实际的行为复杂性集中在少数高密度文件中：`query.ts`（1,700 行，代理循环）、`hooks.ts`（4,900 行，生命周期拦截系统）、`REPL.tsx`（5,000 行，交互式编排器）和记忆系统的提示构建函数。

复杂性来自三个来源，每个都有不同的特征：

协议多样性。 支持五种终端键盘协议、八种 MCP 传输类型、四种远程执行拓扑和七种配置范围本质上是复杂的。每个额外协议是代码库的线性增加，不是指数增加——但总和很大。这种复杂性在 Brooks 意义上是偶然的：它来自环境（终端碎片化、MCP 传输演化、远程部署拓扑），而不是来自要解决的问题。

性能优化。 基于池的渲染、位图搜索预过滤器、粘性缓存锁存和推测性工具执行每个都以可测量的性能收益换取复杂性。这种复杂性由测量证明——每个优化之前都有识别瓶颈的分析数据。风险是优化累积并以使热路径更难修改的方式交互。

行为调优。 记忆系统的提示指令、陈旧警告、验证协议、"忽略记忆"反模式指令——这些不是代码复杂性。它们是提示复杂性，它们带来不同的维护负担。当模型行为在版本间变化时，通过评估仔细调整的提示指令可能需要重新调整。评估基础设施（在代码库中作为案例编号和评估分数引用）是对回归的防御，但它需要持续投资。

该系统的维护负担是显著的。阅读代码库的新工程师不仅要理解代码路径，还要理解驱动特定提示措辞的评估结果、驱动特定安全检查的生产事件，以及驱动特定优化的性能配置文件。代码注释是详尽的——许多包括评估案例编号和前后测量——但近两千个文件中的详尽注释本身就是阅读负担。

代理系统的发展方向

从 Claude Code 的模式中可以看到四个趋势，它们指向该领域的发展方向。

MCP 作为通用协议

第15章将 Claude Code 描述为最完整的 MCP 客户端之一。意义不在于 Claude Code 的实现——而在于 MCP 的存在。工具发现和调用的标准化协议意味着为一个代理构建的工具适用于任何代理。生态系统效应很明显：Postgres 的 MCP 服务器一旦构建，就为每个会说 MCP 的代理服务。开发者在工具集成上的投资是可移植的。

对代理构建者的启示：如果你正在定义自定义工具协议，你可能在犯错误。MCP 足够好，它在变得更好，标准协议的生态系统优势随时间复合。构建一个 MCP 客户端，为规范做贡献，让协议通过社区反馈演化。

多代理协调

Claude Code 的子代理系统（第8章）、任务协调（第10章）和 fork 机制（第9章）是多代理模式的早期实现。它们解决特定问题——缓存共享、并行探索、结构化验证——但它们也揭示了根本性挑战：协调开销。

代理之间的每条消息都消耗令牌。每个 fork 共享缓存但添加父代最终必须协调的对话分支。任务系统的状态机（排队、运行、完成、失败、取消）是添加复杂性而不添加能力的协调机制。随着代理变得更强大，压力将从"我们如何协调多个代理？"转移到"我们如何使一个代理足够强大以至于协调是不必要的？"

当前证据表明两种方法将共存。简单任务将使用单代理。复杂任务将使用协调的多代理系统。工程挑战是使协调开销足够低，以至于交叉点有利于真正并行工作的多代理，而不仅仅是因为任务复杂。

持久记忆

Claude Code 的记忆系统是持久代理记忆的版本 1。基于文件的设计、四类型分类法、LLM 驱动的回忆、陈旧系统和用于长会话的 KAIROS 模式都是第一代解决方案，用于将显著演化的问题。

未来的记忆系统可能会添加结构化检索（当前系统检索整个文件；未来系统可能检索特定事实）、跨项目迁移学习（适用于任何地方的用户偏好，不适用于特定项目）和协作记忆（第11章的团队记忆是第一步，但同步、冲突解决和访问控制是最小的）。

开放问题是基于文件的方法是否可扩展。每个项目 200 个记忆时，它工作。每个项目 2,000 个记忆时，Sonnet 副查询的清单变得太大，合并变得太昂贵，索引超过其上限。随着使用增长，文件优于数据库的架构赌注将面临最艰难的考验。

自主操作

KAIROS 模式、后台记忆提取代理、自动梦境合并、推测性工具执行——这些都是走向自主操作的步骤。代理在没有被要求的情况下做有用的工作：它记住你忘记告诉它要记住的东西，它在睡觉时合并自己的知识，它在当前响应完成前开始执行下一个工具。

轨迹是清晰的。未来的代理将更少反应性，更多主动性。它们会注意到用户没有描述的模式，建议用户没有请求的更正，并在没有显式 `/remember` 命令的情况下维护自己的知识。Claude Code 的记忆系统，带有后台提取安全网和提示工程的"保存什么"启发式，是这种未来的原型。

约束是信任。自主操作需要用户信任代理在无人值守时会做正确的事情。基于文件的记忆、可观察的钩子系统、陈旧警告、权限对话框——所有这些存在是因为信任必须赢得，而不是假设。通往更自主代理的路径经过更透明的代理。

结语

十七章。六个核心抽象。中心的生成器循环，向外扩展的工具，向后延伸的记忆，守卫边界的钩子，将所有内容转换为屏幕上字符的渲染引擎，以及将其连接到代码库之外世界的 MCP。

Claude Code 中最深层次的模式不是任何单一技术。它是将复杂性推到边界的反复决定。渲染系统将复杂性推到池和差异——管道内部，一切都是整数比较。输入系统将复杂性推到分词器和快捷键解析器——处理器内部，一切都是类型化动作。记忆系统将复杂性推到写入协议和回忆选择器——对话内部，一切都是上下文。代理循环将复杂性推到终止状态和工具系统——循环内部，它只是：流式传输、收集、执行、追加、重复。

每个边界吸收混乱并输出秩序。原始字节变成 `ParsedKey`。Markdown 文件变成回忆的记忆。MCP JSON-RPC 变成 `Tool` 对象。钩子退出码变成权限决策。在每个边界的另一侧，世界是混乱的——五种键盘协议、脆弱的 OAuth 服务器、陈旧的记忆、不受信任的仓库钩子。在另一侧，世界是类型化的、有界的、详尽处理的。

如果你正在构建代理系统，这是可迁移的教训。不是具体技术——你可能不需要基于池的渲染或 KAIROS 模式或八种 MCP 传输。而是原则：定义你的边界，在那里吸收复杂性，保持它们之间的一切干净。边界是工程艰难的地方。内部是工程愉快的地方。为愉快的内部设计，将复杂性预算投资在边缘。

源代码是开放的。螃蟹的爪子里有地图。去读吧。